

RED TEAM LEARNING PATH

MODULE 03

HTTP & Web Fundamentals for Red Teamers

Fokus modul

- Membaca HTTP request/response sebagai data terstruktur, bukan sekadar halaman browser.
- Memahami methods, status codes, headers, content types, URLs, query strings, forms, JSON, redirects, cookies, sessions, dan caching basics.
- Membedakan authentication, session management, dan authorization sebagai tiga kontrol yang berbeda.
- Menggunakan curl, jq, Browser DevTools, Burp Suite Community, Python, dan OWASP Juice Shop pada target lokal/authorized.
- Membangun kebiasaan evidence-first sebelum masuk SQL injection, XSS, IDOR, SSRF, API testing, dan vulnerability class lain.

Repository: github.com/VaxXinE/red-teaming

Versi 1.0 - 04 Oktober 2026

Cara Menggunakan Modul Ini

Module 02 mengajarkan bagaimana packet sampai ke service. Module 03 melanjutkan satu layer ke atas: apa yang sebenarnya dikirim aplikasi web di dalam koneksi itu. Jangan langsung mengejar payload. Target modul ini adalah membuat lo mampu melihat satu request dan menjelaskan method, resource, headers, state, identity, serta response tanpa bantuan scanner.

0.1 Learning Objectives

- [] Menjelaskan posisi HTTP di atas TCP/TLS dan sifat request-response.
- [] Membaca URL menjadi scheme, host, port, path, query, dan fragment.
- [] Membedakan GET, HEAD, POST, PUT, PATCH, DELETE, OPTIONS dari sisi semantics.
- [] Membaca class status 1xx-5xx dan mengenali status yang sering muncul saat testing.
- [] Menjelaskan Host, Content-Type, Accept, Authorization, Cookie, Set-Cookie, Location, Origin, Referer, dan cache-related headers.
- [] Menggunakan curl untuk membuat, mengulang, dan membandingkan HTTP requests.
- [] Menggunakan Browser DevTools Network untuk membaca request/response dan storage.
- [] Membedakan authentication, session management, dan authorization.
- [] Menjelaskan Secure, HttpOnly, SameSite, Path, dan Domain pada cookies.
- [] Menggunakan Burp Proxy HTTP history dan Repeater pada local/authorized target.

LEGAL / SCOPE GATE

Seluruh lab utama module ini berjalan pada 127.0.0.1 atau container lokal. Jangan mengubah target curl/Burp menjadi website pihak lain untuk testing aktif tanpa authorization. Normal browsing dokumentasi publik berbeda dari security testing.

0.2 Tooling di Omarchy

Tool	Peran
curl	HTTP client dari terminal; melihat headers, redirects, cookies, body, dan request options.
jq	Membaca dan memformat JSON.
Browser DevTools	Network, Storage/Application, Console, dan request inspection.
Python 3	Menjalankan localhost-only HTTP lab tanpa dependency eksternal.
Docker	Menjalankan OWASP Juice Shop secara terisolasi.
Burp Suite Community	Intercepting proxy, HTTP history, Repeater, Decoder, dan manual web workflow.

Package resmi yang berguna untuk modul ini

```
omarchy pkg add curl jq openssl
```

Burp Suite Community sebaiknya diunduh dari PortSwigger resmi. Untuk modul ini kita tidak bergantung pada package AUR. Pilih installer Linux sesuai arsitektur, bandingkan SHA-256 dengan hash yang ditampilkan di halaman download, lalu jalankan installer.

Verifikasi installer sebelum menjalankan

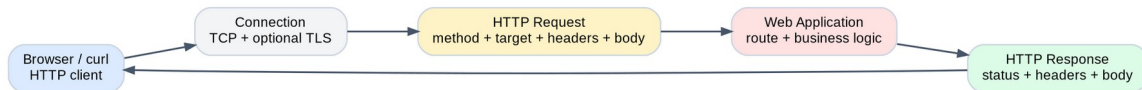
```
uname -m
sha256sum <nama-installer-burp>
chmod +x <nama-installer-burp>
./<nama-installer-burp>
```

AUR awareness

Omarchy memudahkan instalasi AUR, tetapi AUR tidak divalidasi oleh Arch team. Untuk tool security yang sensitif seperti proxy, prioritaskan vendor resmi atau review PKGBUILD dan upstream source terlebih dahulu.

1. Mental Model HTTP

HTTP adalah protocol aplikasi dengan model request-response. Client meminta representasi atau tindakan terhadap suatu resource; server membalas status, metadata, dan optional content. Pada HTTPS, HTTP tetap konsep yang sama tetapi bytes-nya dilindungi TLS selama transit.



Gambar 1 - Mental model sederhana request-response HTTP di atas connection layer.

Think Like an Attacker

Saat melihat request, tanyakan: bagian mana yang dikontrol client? Bagian mana yang dianggap trusted oleh server? Identity/state dibawa lewat apa? Resource apa yang dipilih? Keputusan authorization terjadi di mana?

1.1 HTTP/1.1, HTTP/2, HTTP/3 - apa yang perlu dipahami sekarang?

- HTTP/1.1 sering mudah dibaca sebagai teks dan sangat bagus untuk belajar anatomy request/response.
- HTTP/2 membawa konsep HTTP yang sama tetapi framing/multiplexing berbeda di wire.
- HTTP/3 berjalan di atas QUIC/UDP. Untuk module ini fokus ke semantics HTTP, bukan internal framing protocol.
- Burp dan DevTools akan menormalisasi tampilan sehingga lo tetap bisa belajar method, URL, headers, parameters, body, dan response.

1.2 Stateless protocol dan application state

HTTP tidak secara otomatis mengingat bahwa dua request berasal dari user yang sama. Web application membangun state menggunakan session identifiers, cookies, tokens, database state, atau mekanisme lain. Karena itu session identifier adalah security-sensitive artifact.

2. URL dan Request Target

```
https://shop.example.test:8443/products/42?currency=IDR&view=full#reviews
```

Komponen	Contoh	Makna
Scheme	https	Protocol scheme; biasanya HTTP atau HTTPS.
Host	shop.example.test	Nama host yang ingin diakses.
Port	8443	Port explicit. HTTPS default biasanya 443.
Path	/products/42	Resource path yang dipilih.
Query	currency=IDR&view=full	Data setelah ? yang dikirim ke server.
Fragment	#reviews	Dipakai client/browser; fragment tidak dikirim sebagai HTTP request target ke server.

Security perspective

Parameter bukan hanya query string. Input dapat muncul di path segment, headers, cookies, body form, JSON, multipart upload, GraphQL variables, atau nested objects. Nanti enumeration web harus mencari semua input surface tersebut.

3. Anatomy of an HTTP Request

```
GET /api/profile?view=compact HTTP/1.1
Host: 127.0.0.1:8083
Accept: application/json
User-Agent: curl/8.x
Cookie: lab_session=demo-only
```

- Request line: method + target + protocol version (pada representasi HTTP/1.1).
- Headers: metadata dan control information.
- Blank line: pemisah headers dengan body.
- Body: optional; umum pada POST/PUT/PATCH, tetapi semantics ditentukan method dan application.

3.1 Methods

Method	Intent umum	Catatan security/testing
GET	Retrieve representation	Harus diperlakukan sebagai read semantics; query sering membawa user-controlled input.
HEAD	GET tanpa response content	Berguna melihat metadata/headers.
POST	Submit data / process action	Sering membawa form/JSON dan state-changing operation.
PUT	Create/replace resource representation	Biasanya idempotent secara semantics.
PATCH	Partial modification	Format body bergantung API.
DELETE	Remove resource	Authorization sangat penting.
OPTIONS	Capabilities/options	Dapat menunjukkan allowed methods; juga muncul pada CORS preflight.

Do not over-trust method names

Server yang buruk bisa melakukan destructive action lewat GET atau mengabaikan method semantics. Tester harus mengamati behavior aktual, tetapi tetap jangan melakukan state-changing tests di luar authorized lab.

4. Anatomy of an HTTP Response

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 53
X-Lab-Module: 03

{"user":"student","role":"learner","module":3}
```

4.1 Status code classes

Class	Makna	Contoh yang sering terlihat
1xx	Informational	100 Continue
2xx	Successful	200 OK, 201 Created, 204 No Content

3xx	Redirection	301, 302, 307, 308
4xx	Client-side request problem	400, 401, 403, 404, 405, 429
5xx	Server failed to fulfill valid-looking request	500, 502, 503

401 vs 403

Secara practical, 401 biasanya berkaitan dengan authentication requirement/challenge, sedangkan 403 menunjukkan server memahami request tetapi menolak akses. Real applications kadang memakai status secara tidak konsisten, jadi lihat behavior dan headers juga.

5. Headers yang Wajib Lo Kenal

Header	Direction	Mengapa penting
Host	Request	Virtual host / authority target.
User-Agent	Request	Client metadata; jangan dipercaya sebagai identity.
Accept	Request	Representation yang diinginkan client.
Content-Type	Both	Media type body, mis. application/json atau form encoded.
Content-Length	Both	Ukuran payload pada representation tertentu.
Authorization	Request	Carries auth credentials/token scheme; sensitive.
Cookie	Request	Cookies yang client kirim.
Set-Cookie	Response	Server meminta browser menyimpan cookie.
Location	Response	Target redirect atau created resource.
Origin	Request	Origin initiating request; penting pada CORS/CSRF context.
Referer	Request	Source page context; bisa dibatasi oleh Referrer-Policy.
Cache-Control	Both	Caching directives.

5.1 Content types

```
Content-Type: application/json
Content-Type: application/x-www-form-urlencoded
Content-Type: multipart/form-data; boundary=...
Content-Type: text/html; charset=utf-8
```

Common mistake

Jangan menyimpulkan input handling hanya dari tampilan UI. Request bisa dikirim dengan content type berbeda, dan server-side parser yang berbeda dapat menghasilkan behavior berbeda. Kita baru akan menguji class vulnerability di module berikutnya.

6. Forms, JSON, dan Encoding

Browser forms tradisional sering mengirim application/x-www-form-urlencoded atau multipart/form-data. Modern frontend banyak berkomunikasi lewat JSON API. Dalam testing, bedakan format data dari transport: semua tetap HTTP request.

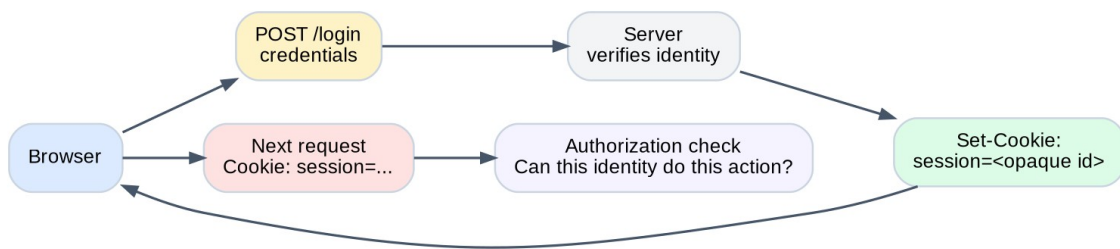
```
# form encoded
curl -i -X POST http://127.0.0.1:8083/echo \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'name=arya&role=student'

# JSON
curl -i -X POST http://127.0.0.1:8083/echo \
  -H 'Content-Type: application/json' \
  --data '{"name":"arya","role":"student"}'
```

Output encoding != input validation

Input validation membatasi data yang diterima. Output encoding memastikan data diperlakukan sebagai data di context output tertentu. Keduanya berbeda dan akan sangat penting saat membahas XSS/injection nanti.

7. Cookies & Sessions



Gambar 2 - Authentication, session identifier, dan authorization harus dipahami sebagai tahap berbeda.

Cookie adalah mekanisme browser untuk menyimpan name/value dan mengirimkannya kembali sesuai scope. Session adalah konsep application state; session identifier sering dibawa oleh cookie. Jangan menyamakan "cookie" dengan "session" secara mutlak.

7.1 Cookie attributes

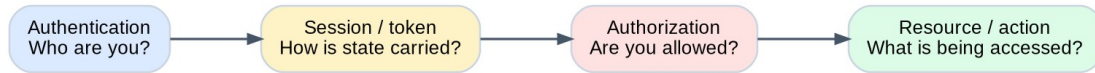
Attribute	Tujuan
Secure	Kirim cookie hanya lewat secure HTTPS context, dengan localhost exception pada browser tertentu.
HttpOnly	Mencegah akses cookie melalui JavaScript Document.cookie; defense-in-depth terhadap theft via XSS.
SameSite	Mengontrol pengiriman cookie pada cross-site requests; membantu mitigasi beberapa CSRF scenarios.
Path	Membatasi request path yang menerima cookie; bukan authorization boundary.
Domain	Memperluas scope host/subdomain; gunakan seketat mungkin.
Max-Age / Expires	Mengatur lifetime/persistence.

```
Set-Cookie: __Host-session=<opaque>; Path=/; Secure; HttpOnly; SameSite=Lax
```

Session token = temporary credential

Setelah login, session ID/token dapat menjadi ekuivalen sementara dari authentication yang sudah dilakukan. Perlakukan capture session sebagai credential. Jangan pernah commit session token real ke public repository atau screenshot report tanpa redaction.

8. Authentication vs Authorization



Gambar 3 - Identity verification, state, dan permission decision adalah concerns berbeda.

Konsep	Pertanyaan	Contoh
Authentication	Siapa lo?	Password, WebAuthn, SSO, certificate.
Session Management	Bagaimana server mengenali lo pada request berikutnya?	Opaque session cookie, access token.
Authorization	Boleh melakukan apa terhadap resource ini?	User A boleh membaca invoice A, tetapi tidak invoice B.

Red Team perspective

Banyak bug serius bukan kegagalan login, tetapi kegagalan authorization setelah login. Nanti saat masuk IDOR/BOLA, pertanyaannya bukan "bisa login?" melainkan "server benar-benar memeriksa siapa yang boleh mengakses object/action ini?"

9. Redirects, Caching, Origin & CORS Basics

9.1 Redirects

3xx response biasanya membawa Location. curl secara default menunjukkan response awal; -L mengikuti redirects. Saat testing, catat setiap hop karena cookie, scheme, host, dan authorization context dapat berubah.

```
curl -i http://127.0.0.1:8083/redirect
curl -i -L http://127.0.0.1:8083/redirect
```

9.2 Cache basics

Caching dapat terjadi di browser, reverse proxy, CDN, atau shared intermediary. Header seperti Cache-Control, ETag, Last-Modified, Vary, dan Age membantu menjelaskan cache behavior. Module ini hanya mengenalkan konsep; cache poisoning masuk tahap advanced.

9.3 Same-Origin Policy dan CORS

Origin ditentukan oleh scheme, host, dan port. Browser menerapkan Same-Origin Policy untuk membatasi interaksi tertentu antar-origin. CORS adalah mekanisme berbasis response headers yang memberi browser izin terkontrol untuk cross-origin access. CORS bukan pengganti authorization.

```
Origin A: https://app.example.test:443
Origin B: https://api.example.test:443    # host berbeda
Origin C: http://app.example.test:80      # scheme/port berbeda
```

Security note

Kalau API mengembalikan data sensitif tanpa authorization, "CORS ketat" tidak memperbaiki broken access control. CORS terutama mengatur apa yang browser frontend boleh baca lintas origin.

10. Hands-on Lab 1 - Start Local HTTP Lab

Objective

Jalankan web server training kecil yang hanya bind ke loopback. Semua request berikutnya aman untuk diulang dan dimodifikasi karena target adalah proses milik lo sendiri.

```
mkdir -p ~/CyberSec/labs/module03
cd ~/CyberSec/labs/module03
cp /path/to/http-lab-server.py .
chmod +x http-lab-server.py
./http-lab-server.py
```

Expected listener: http://127.0.0.1:8083. Di terminal lain:

```
curl -i http://127.0.0.1:8083/
```

- [] Response status terlihat.
- [] Content-Type terlihat.
- [] X-Lab-Module: 03 terlihat.
- [] Server tidak listen di 0.0.0.0.

Why loopback?

127.0.0.1 membuat service hanya dapat diakses dari host lokal. Ini default yang lebih aman untuk training service dibanding publish ke seluruh LAN.

11. Hands-on Lab 2 - Read Request/Response with curl

```
curl -v http://127.0.0.1:8083/api/profile
curl -sS -D headers.txt http://127.0.0.1:8083/api/profile -o body.json
cat headers.txt
jq . body.json
```

Tugas: identifikasi method, Host, Accept/User-Agent jika ada, status, Content-Type, Content-Length, dan body. Bedakan output verbose curl dari actual response body.

12. Hands-on Lab 3 - Methods & Bodies

```
curl -i -X HEAD http://127.0.0.1:8083/
curl -i -X OPTIONS http://127.0.0.1:8083/

curl -i -X POST http://127.0.0.1:8083/echo \
-H 'Content-Type: application/json' \
--data '{"message":"hello","module":3}'
```

Common mistake

Untuk HEAD, curl memiliki opsi `-I/--head` yang lebih natural daripada `-X HEAD`. `-X` hanya mengubah method token dan pada command tertentu dapat menghasilkan behavior yang tidak lo maksud. Belajar semantics tool, bukan sekadar memaksa method.

```
curl -I http://127.0.0.1:8083/
```

13. Hands-on Lab 4 - Query Strings & Encoding

```
curl -sS 'http://127.0.0.1:8083/search?q=hello%20world' | jq .  
curl -sS -G http://127.0.0.1:8083/search --data-urlencode 'q=hello world & web' | jq .
```

Bandungkan URL yang terlihat di verbose mode. Perhatikan bahwa percent-encoding adalah representation detail, bukan sanitization atau security control.

14. Hands-on Lab 5 - Redirects

```
curl -i http://127.0.0.1:8083/redirect  
curl -i -L http://127.0.0.1:8083/redirect
```

[] Status response pertama dicatat.

[] Location header dicatat.

[] Final status setelah -L dicatat.

15. Hands-on Lab 6 - Cookies with curl

```
rm -f cookies.txt  
curl -i -c cookies.txt http://127.0.0.1:8083/set-cookie  
cat cookies.txt  
curl -v -b cookies.txt http://127.0.0.1:8083/api/profile
```

`-c` menyimpan cookie jar; `-b` membacanya pada request berikutnya. File cookies.txt pada lab ini hanya berisi demo cookie, tetapi pada target real file cookie jar dapat menjadi credential material.

Credential hygiene

Jangan commit cookies.txt, Authorization headers, JWT real, API keys, atau session captures ke Git. Tambahkan evidence/credential artifacts ke .gitignore dan simpan dengan permission minimum yang diperlukan.

16. Hands-on Lab 7 - Browser DevTools

Objective

Lihat HTTP dari browser tanpa proxy dulu. Ini membuat lo mengerti apa yang browser otomatis tambahkan dibanding request curl manual.

1. Buka `http://127.0.0.1:8083/` di browser.
2. Buka DevTools -> Network. Aktifkan Preserve log hanya jika dibutuhkan.
3. Reload halaman. Pilih request document.
4. Catat Request URL, Method, Status, Remote Address, Request Headers, Response Headers.
5. Buka `http://127.0.0.1:8083/set-cookie` lalu lihat Storage/Application -> Cookies.
6. Verifikasi apakah HttpOnly cookie terlihat pada storage inspector tetapi tidak bisa dibaca lewat `document.cookie`.

Jalankan di Console setelah `/set-cookie`

```
document.cookie
```

Karena lab cookie memakai HttpOnly, ``lab_session`` seharusnya tidak muncul melalui JavaScript `Document.cookie`. Browser tetap dapat mengirimkannya pada request yang matching scope.

17. Hands-on Lab 8 - OWASP Juice Shop Local

OWASP Juice Shop sengaja vulnerable dan cocok untuk training. Pada module ini kita belum mengeksploitasi challenge; kita hanya menggunakannya untuk melihat aplikasi SPA, requests API, headers, dan workflow browser.

```
cd ~/CyberSec/labs/module03
./juice-shop-lab.sh create
./juice-shop-lab.sh status
```

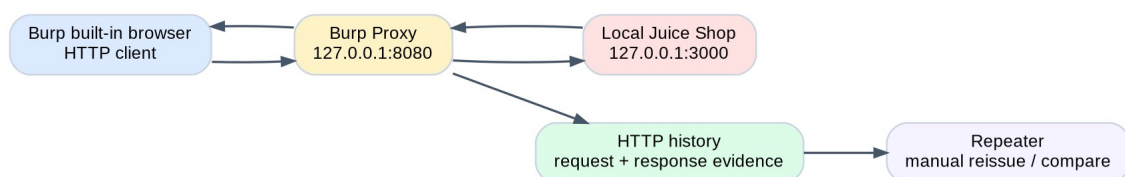
Buka `http://127.0.0.1:3000` dan DevTools -> Network. Browse beberapa halaman normal. Cari request XHR/fetch/API dan jawab:

- [] Endpoint API mana yang dipanggil frontend?
- [] Method apa yang digunakan?
- [] Apakah body JSON digunakan?
- [] Response Content-Type apa?
- [] Ada cookie atau Authorization header?
- [] Status codes apa yang muncul selama normal browsing?

Scope reminder

Jangan gunakan scanner otomatis atau mencoba challenge exploit dulu. Tujuan module ini adalah memahami protocol traffic dan application workflow.

18. Burp Suite Community Fundamentals



Gambar 4 - Burp ditempatkan di antara browser training dan target lokal agar request/response dapat diamati dan diulang.

Burp Proxy mencatat traffic yang melewati proxy. HTTP history adalah evidence ledger praktis: host, method, URL, params, status, MIME type, TLS, cookies, dan metadata lain. Repeater memungkinkan satu request dikirim ulang secara manual untuk membandingkan behavior.

18.1 Install from official source

7. Buka halaman resmi PortSwigger Burp Suite downloads.
8. Pilih Linux build sesuai `uname -m` dan Community Edition saat installer berjalan.
9. Bandingkan SHA-256 file lokal dengan hash yang ditampilkan PortSwigger.
10. Jalankan installer sebagai user biasa. Elevation hanya jika installer memang meminta untuk lokasi system-wide.
11. Start Burp dengan temporary project/default configuration untuk lab.

Supply-chain note

Jangan download Burp dari mirror random, file-sharing site, atau script curl|sh yang tidak diverifikasi. Security tools punya akses ke sensitive traffic; provenance penting.

19. Hands-on Lab 9 - Burp Proxy & HTTP History

12. Pastikan Juice Shop lokal berjalan di 127.0.0.1:3000.
13. Start Burp Suite Community.
14. Proxy -> Intercept: set Intercept off untuk browsing awal agar flow tidak macet.
15. Klik Open browser dari Burp. Built-in browser sudah diarahkan melalui Burp Proxy.
16. Buka http://127.0.0.1:3000 dan browse normal.
17. Kembali ke Proxy -> HTTP history. Filter host 127.0.0.1 / in-scope traffic.
18. Pilih satu GET/API request. Baca request dan response berdampingan.
- [] Method dan path dicatat.
- [] Query/body parameters dicatat.
- [] Request Content-Type dicatat.
- [] Response status/Content-Type dicatat.
- [] Cookies/token hanya dicatat secara redacted jika sensitive.

20. Hands-on Lab 10 - Burp Repeater

Repeater adalah salah satu tool manual terpenting. Pada module ini kita gunakan hanya untuk perubahan harmless pada localhost.

19. Dari HTTP history, pilih request GET ke local HTTP lab atau harmless GET Juice Shop.
20. Right-click -> Send to Repeater.
21. Kirim request tanpa perubahan dan simpan baseline status/length.
22. Jika target adalah /search pada local lab, ubah nilai q dari hello menjadi world.
23. Send lagi. Bandingkan request/response secara manual.
24. Ubah satu hal pada satu waktu. Jangan mengandalkan tebakan acak.

Methodology habit

Baseline -> one controlled change -> observe -> record. Pola ini akan kita pakai terus saat SQLi, XSS, access control, SSRF, API testing, dan authentication testing.

21. TLS/HTTPS Fundamentals

HTTPS adalah HTTP yang dilindungi TLS. TLS memberi confidentiality dan integrity selama transit serta server authentication melalui certificate validation. Ia tidak menjamin bahwa aplikasi bebas SQLi, access-control bug, logic flaw, atau compromised endpoint.

Konsep	Makna praktis
Certificate	Mengikat public key dengan identity/name sesuai trust model certificate.
SNI	Client memberi server name saat TLS negotiation agar server dapat memilih certificate/site.
CA trust	Client memverifikasi chain ke trust anchor yang dipercaya.
Hostname validation	Certificate harus valid untuk hostname target.
TLS termination	TLS bisa berakhir di reverse proxy/load balancer sebelum request diteruskan internal.

Optional read-only observation ke public example.com

```
openssl s_client -connect example.com:443 -servername example.com </dev/null
```

Burp + HTTPS

Untuk external browser, interception HTTPS memerlukan trust terhadap Burp CA khusus lab. Built-in browser Burp sudah dirancang untuk workflow ini. Jangan memasang CA training ke trust store system-wide tanpa kebutuhan; scope trust sekecil mungkin.

22. HTTP Observation Worksheet

Field	Yang harus dicatat
Target	scheme://host:port
Request	method + path + query
Headers	security-relevant + content negotiation
Body	format dan parameter names, redacted bila sensitive
Identity	anonymous / session / token type
Response	status + Content-Type + notable headers
State change	none / expected / unexpected
Next hypothesis	satu pertanyaan yang akan diuji berikutnya

Simpan worksheet per application/target. Jangan menaruh secret real ke repository publik.

23. Mini Project - HTTP Baseline Collector

Buat script kecil yang mengambil endpoint localhost, menyimpan headers dan body terpisah, lalu mencatat hash evidence. Tujuannya reproducibility, bukan scanner.

```
#!/usr/bin/env bash
set -euo pipefail
umask 077

TARGET="${1:-http://127.0.0.1:8083/api/profile}"
case "$TARGET" in
  http://127.0.0.1:*|http://localhost:*) ;;
  *) echo "Refusing non-local target" >&2; exit 2 ;;
esac

STAMP="$(date +%Y%m%d-%H%M%S)"
OUT="http-baseline-$STAMP"
mkdir -m 700 "$OUT"

curl -sS -D "$OUT/headers.txt" "$TARGET" -o "$OUT/body.bin"
sha256sum "$OUT/headers.txt" "$OUT/body.bin" > "$OUT/SHA256SUMS"
printf 'target=%s\n' "$TARGET" > "$OUT/metadata.txt"

echo "Saved to $OUT"
```

Production habit

Evidence should be reproducible, timestamped, minimally exposed, and separated from code. Hashes help show whether captured artifacts changed after collection.

24. Common Mistakes

Mistake	Perbaikan
Hanya melihat UI	Baca request/response di DevTools/Burp.
Menghafal status code tanpa behavior	Interpret status bersama headers, body, redirect chain, dan application state.
Menganggap cookie = auth	Pisahkan cookie mechanism, session state, authentication, dan authorization.
Menganggap HTTPS = aplikasi aman	TLS melindungi transport, bukan business logic.
Menyimpan token di notes/Git	Redact dan isolasi credential artifacts.
Burp Intercept selalu ON lalu bingung browser macet	Mulai Intercept off; gunakan HTTP history untuk observasi.
Mengubah banyak parameter sekaligus	Baseline dan ubah satu variable pada satu waktu.
Testing domain publik tanpa izin	Gunakan local lab/authorized training platform.

25. Knowledge Check

1. Apa beda HTTP dan HTTPS?
2. Bagian URL mana yang tidak dikirim sebagai request target ke server?
3. Apa fungsi Host header?
4. Mengapa method semantics tetap penting walau server bisa salah implementasi?
5. Bedakan 2xx, 3xx, 4xx, dan 5xx.
6. Apa beda Content-Type dan Accept?
7. Kapan application/json berbeda dari form-urlencoded?
8. Apa beda Cookie dan Set-Cookie?
9. Apa fungsi Secure cookie flag?
10. Apa fungsi HttpOnly?
11. Apa tujuan SameSite?

12. Kenapa Path bukan authorization control?
13. Bedakan authentication, session management, dan authorization.
14. Mengapa session token harus diperlakukan seperti credential?
15. Apa itu origin?
16. CORS menyelesaikan authorization bug atau tidak? Jelaskan.
17. Apa perbedaan curl -i dan -v?
18. Mengapa Burp HTTP history berguna meski Intercept off?
19. Apa fungsi Repeater?
20. Kenapa tester harus punya baseline sebelum memodifikasi request?

26. Practical Challenge - 35 Menit

Scenario

Lo menerima localhost HTTP lab dan Juice Shop lokal sebagai authorized targets. Buat protocol map tanpa mengeksploitasi vulnerability.

25. Start http-lab-server.py dan Juice Shop local.
 26. Dengan curl, capture /api/profile headers + JSON body dan format dengan jq.
 27. Tunjukkan redirect chain /redirect tanpa dan dengan -L.
 28. Set demo cookie ke cookie jar dan buktikan cookie dikirim kembali.
 29. Di DevTools, identifikasi minimal 3 requests berbeda dari Juice Shop.
 30. Di Burp HTTP history, pilih satu request API dan catat method, path, headers penting, status, content type.
 31. Send satu harmless request ke Repeater dan ubah satu parameter non-destructive.
 32. Buat HTTP Observation Worksheet untuk dua endpoints.
 33. Cleanup container dan hapus cookie jar/evidence yang tidak perlu.
- Deliverable minimum: notes.md + screenshot redacted bila perlu + evidence directory lokal yang tidak di-commit.

27. Exit Criteria

- [] Gue bisa membaca raw HTTP request dan response tanpa bergantung pada UI browser.
- [] Gue paham URL components, methods, status codes, headers, content types, forms, JSON, redirects, cookies, dan sessions.
- [] Gue bisa menjelaskan authentication vs session vs authorization.
- [] Gue bisa memakai curl dan jq untuk mereproduksi request lokal.
- [] Gue bisa membaca Browser DevTools Network/Storage.
- [] Gue bisa menjalankan Juice Shop hanya pada localhost.
- [] Gue bisa menggunakan Burp HTTP history dan Repeater pada target authorized.
- [] Gue tahu bahwa token/session evidence adalah sensitive dan tidak boleh masuk public Git.

Next module

Module 04 adalah Windows Fundamentals. Setelah fondasi OS selesai, kita kembali ke web di Module 10-13 untuk Burp dan vulnerability classes secara jauh lebih dalam.

28. Cheat Sheet

```
# Basic
curl -i http://127.0.0.1:8083/
curl -v http://127.0.0.1:8083/api/profile
curl -I http://127.0.0.1:8083/

# Save headers/body
curl -sS -D headers.txt URL -o body.bin

# JSON
curl -sS URL | jq .

# POST JSON
curl -i -X POST URL \
  -H 'Content-Type: application/json' \
  --data '{"key":"value"}'

# Query safely encoded
curl -G URL --data-urlencode 'q=hello world'

# Redirects
curl -i URL
curl -i -L URL

# Cookie jar
curl -c cookies.txt URL
curl -b cookies.txt URL

# Local listener check
ss -ltnp | grep 8083
```

29. Referensi

RFC 9110 - HTTP Semantics - Methods, status codes, representation semantics, dan core HTTP concepts.

<https://www.rfc-editor.org/rfc/rfc9110.html>

MDN - HTTP Overview - Practical HTTP model dan web platform context.

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview>

MDN - Using HTTP cookies - Secure, HttpOnly, SameSite, cookie scope dan security considerations.

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies>

OWASP Authentication Cheat Sheet - Authentication design dan reauthentication guidance.

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

OWASP Session Management Cheat Sheet - Session identifiers, lifecycle, transport, dan session security.

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

OWASP Authorization Cheat Sheet - Distinction dan design principles untuk authorization.

https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

OWASP Juice Shop Companion Guide - Running - Local Docker deployment dengan loopback binding.

<https://pwning.owasp-juice.shop/companion-guide/local/part1/running.html>

PortSwigger - Download and install Burp Suite - Official installation workflow untuk Community/Professional.

<https://portswigger.net/burp/documentation/desktop/getting-started/download-and-install>

PortSwigger - HTTP history - HTTP history fields dan workflow.

<https://portswigger.net/burp/documentation/desktop/tools/proxy/http-history>

PortSwigger - Reissue requests with Repeater - Manual request reissue workflow.

<https://portswigger.net/burp/documentation/desktop/getting-started/reissuing-http-requests>

Omarchy Manual - Other Packages - omarchy pkg add/drop dan AUR trust warning.

<https://omarchy.org/manual/other-packages/>

Appendix A - Lab Cleanup

```
# Stop Python local server with Ctrl+C

# Juice Shop
cd ~/CyberSec/labs/module03
./juice-shop-lab.sh cleanup

# Remove temporary demo artifacts if no longer needed
rm -f cookies.txt headers.txt body.json

# Verify listeners are gone
ss -ltnp | grep -E ':(8083|3000)\b' || true
```

Cleanup principle: training service tidak seharusnya terus hidup tanpa alasan. Selesai lab: stop process/container, verifikasi listener hilang, dan bersihkan sensitive artifacts.